Tidal Wave: Objective, Scalable Block Template Construction over a Semi-Persistent Bitcoin Transaction Pool

Eric Voskuil
eric@voskuil.org*

12 November 2026

Abstract

A companion paper [1] describes a transaction pool that admits every conflicting transaction that pays for its own block space and never chooses among them. This paper describes what a miner does with such a pool. A block template is a choice of one outcome per conflict component and a packing of the chosen outcomes, together with their uncontested ancestry, into a block's weight and signature-operation limits less a reserve for the coinbase. We show that the choice is independent across components because the pool's labels partition it, that the packing is a greedy over *chunks*, ancestry-closed sets scored by their own fee rate, and that a chunk's fee never pays for bytes in a block other than its own because a chunk is split by prefix at the block limit. The miner's criterion for choosing among outcomes, and among chunks, is a parameter of the computation rather than of the pool. The template is derived from the pool's columns by a small number of whole-column passes, with no maintained per-transaction ordering state, in the same two forms as admission: a frontier walk on a CPU and kernels on SIMD lanes or a GPU. Because row order in the pool is a topological order, the template's transaction order is read rather than computed. We compare the construction with bitcoind's ancestor-set selection and with the cluster mempool's chunk merge, from which it differs in where conflicts are resolved, where ordering is produced and what state is kept between templates, and we estimate template cost over a pool of one million rows at three levels of pool structure.

1 Introduction

A block template is a set of unconfirmed transactions, in an order that places every parent before its children, that fits within a block's limits and leaves room for the coinbase. A miner wants the template that earns the most fee, or the template that best serves some other objective of the miner's own, and wants it quickly, because a template built against a stale chain top is worthless and a template built slowly after a new block is lost revenue.

In bitcoind the template is drawn from a mempool that is conflict-free by construction, so the miner's problem is a packing problem over a dependency graph, and the mempool maintains the state the packing needs. In libbitcoin the tx pool holds conflicting transactions deliberately [1]. The pool's admission rule guarantees that the best conflict-free outcome of every conflict component pays for every byte in it, and it refuses to decide which outcome that is. The miner's problem therefore has one more step than bitcoind's: before packing, the miner resolves each component to one outcome. That step is where the miner's incentive enters, and it is the step the pool exists to defer.

*Libbitcoin. This paper was drafted from the design sessions of 12 November 2026 with the assistance of Claude; the design, its history and all decisions recorded here are the author's. Cost figures are estimates from the stated models and are not measurements.

This paper is about those two steps and the computation that performs them. Its argument is that resolution and packing are the same kind of work as admission, column passes over the pool's persistent index, and that they are cheap enough to be rerun from nothing at every chain top rather than maintained as state. The 2021 node design ranked block template deduction speed third among its optimization goals, after denial-of-service protection and restartability [2]. The pool paper answered the first; this one answers the third.

The three aspects of unconfirmed transaction handling, relay, pool and mining, share one unit, a set that fits a block, and one accounting, fee against chargeable bytes [1, 3]. Relay forms a package for a peer by walking unmarked ancestry until the aggregate clears the peer's rate. Mining forms a chunk for a block by walking untaken ancestry until the aggregate is scored. The walk is the same, the marks are the same kind of head, and the criterion differs: the peer's rate for relay, the miner's objective for mining.

2 History of the design

The version 4 node design [2] describes the transaction pool as validated transactions awaiting confirmation, double spends included, with order imposed only by blocks: "Ordering transactions to resolve double spends is the reason that blocks exist." It anticipates a transaction metadata graph for template construction in a later version, and hardware acceleration of its processing. The version 4 pool is flat, recording per transaction its validation context, fee, size, sigops, parents and time, and the companion paper describes the persistent label index that version 4 adds to it: cells, children, edges and totals keyed by pool row, and rows, siblings and rivals keyed by input [1].

Template construction was left as an open item there, with its shape fixed: a choice of one outcome per component and a packing of the chosen outcomes with their uncontested ancestry into the block's limits, run as whole-column passes once per template rather than once per admission, with the miner's criterion as a parameter. This paper develops that item. It adds no state to the pool. Everything a template needs is in the index already, and what the version 5 metadata graph adds is the object model through which a miner expresses the choice, not the data over which it is made.

3 Model

We take the model of the companion paper as given and restate only what the template needs. The tx pool table has one row per pooled transaction. A row's parents are the transactions whose outputs it spends, each either another pool row or a confirmed transaction, and a confirmed transaction is a *boundary*: a parent, never a member. A point spent by two or more rows is *contested* and its spenders form a *clique*. The *contested closure* is the clique members and all of their pool descendants; a *component* is a maximal connected set of closure rows; an *outcome* of a component is a choice of one winner per clique with the losers and their descendants removed, and is conflict-free and ancestry-closed within the closure. A row's *chargeable size* is $s(r) = \max(v(r), \beta\sigma(r))$ for virtual size v, sigops σ and configured bytes per sigop β.

3.1 The uncontested rows

Every pool row is either in exactly one component or in none. A row in no component is *uncontested*: it spends only boundaries and uncontested rows, and nothing contested descends to it. Uncontested rows form a forest of chains and trees, unbounded in depth by the pool, whose fee is the same in every outcome of every component and therefore plays no part in admission. It plays every part in mining. A miner cannot include an outcome without including its ancestors, and an outcome's ancestors outside the closure are uncontested rows.

Definition 3.1 (Ancestry)**.** *For a set S of rows,* anc(S) *is S together with every pool row from which a member of S descends. Boundaries are not included. A set is* closed *when* anc(S) $= S$*.*

A block is a closed conflict-free set, and a template is a block less its coinbase.

3.2 Row order is topological

The pool is ordered by entry. A transaction is admitted only when every parent is confirmed or already pooled, or is admitted ahead of it within the same set, and its row is allocated at admission. Row order is therefore a topological order of the pool graph.

Lemma 3.1 (Row order)**.** *If p is a pool parent of r then p's row precedes r's row.*

The lemma is the reason the template never sorts transactions. Any closed set taken from the pool, listed in ascending row order, satisfies the block's parent-before-child requirement. Ordering, which bitcoind produces by linearization and maintains per cluster, is here a property of the table.

3.3 Limits

A block has four million weight units, which is one million virtual bytes, and eighty thousand signature operations [8]. The node reserves a configured weight and sigop allowance for the coinbase, including its witness commitment, and the template's budget is the block less that reserve. The companion paper caps a contested component's chargeable size, members plus uncontested ancestry, at the same budget, so every outcome of every component fits a template on its own. No such cap applies to uncontested rows; a chain of any length is admitted, and it is the template's job to take as much of it as fits.

4 Resolution

4.1 One outcome per component

The pool holds every outcome of a component and credits the component with the fee of its best. The miner takes one. Which one is the miner's to say, and the computation that says it is the same enumeration the admission rule runs: outcomes are decoded from an index by mixed radix over the cliques, each is a mask over the component's members, and each is scored.

Definition 4.1 (Resolution)**.** *A* resolution *of the pool is a choice of one outcome per component. The* included set *of a resolution is the union of the chosen outcomes together with the uncontested rows. The* excluded set *is the complement: every loser and every descendant of a loser.*

The included set is conflict-free, since each outcome is conflict-free and no two components share a contested point, and it is closed, since each outcome is closed within its closure and the closure's ancestors outside it are uncontested rows, all of which are included. Resolution turns the pool into exactly the kind of object bitcoind's mempool is by construction: a conflict-free dependency graph. Everything after resolution is a packing problem over that graph.

4.2 Independence

Proposition 4.1 (Independence)**.** *The choice of outcome in one component does not change the fee, chargeable size or ancestry of any outcome in another.*

Components are disjoint and no edge passes between them except through boundaries and uncontested rows, which belong to every outcome. So the resolution of the pool factors into the resolution of its components, each of which is one bounded enumeration, and all of which proceed at once. On the GPU each component is a thread block; on the CPU each is a task. The labels that partition the pool for admission partition it for mining.

4.3 The criterion

What "best" means is the miner's. The obvious criterion is fee: take the outcome with the greatest fee, which is also what the pool credits the component with, so that under the fee criterion the pool's invariant and the miner's revenue coincide. A second natural criterion is rate, fee over chargeable size including the outcome's uncontested ancestry, which prefers compact outcomes when block space is scarce. The pool makes no claim about which is right. The criterion is a function of the outcome's fee, size and members, evaluated inside the enumeration, and nothing downstream depends on which function it is. A miner who prefers its own transactions, or the earlier-seen contender, or the contender a customer paid it to prefer, expresses that here, and the pool is indifferent.

Two observations make the criterion harmless to the rest of the construction. First, any outcome of a component fits the template budget, by the component cap, so no choice can produce an unpackable unit. Second, the packing that follows is a greedy over rates, so a fee-maximal outcome that is large is not thereby guaranteed a place; it competes for space like everything else. Resolution says which contender is eligible. Packing says whether it is taken.

5 Chunks

After resolution the included set is a forest of dependency trees and DAGs over which the template is packed. The unit of packing is the chunk.

5.1 Definition

Definition 5.1 (Chunk)**.** *For an included row r and a set T of rows already taken, the* chunk *of r is* $\mathrm{anc}(\{r\}) \setminus T$*, the untaken ancestry of r including r. Its fee and chargeable size are the sums over its rows, and its* rate *is fee over chargeable size.*

A chunk is closed relative to what has been taken: adding it to T yields a closed set. The chunk of a row with no untaken pool ancestors is the row alone. The chunk of a leaf of a long chain is the whole untaken chain. This is the ancestor set of ancestor-set mining [4], and the construction here is a greedy over chunk rates for the same reason bitcoind's is: a child cannot pay for block space without its parents in the block, so the unit of value is the closed set, not the transaction.

5.2 Depth

A chunk's size is unbounded in principle, because uncontested chains are unbounded. A configured *chunk depth* D bounds the walk: a chunk is formed only over ancestors within D parent steps of its root, and a root whose untaken ancestry reaches past D is not *eligible* until enough of its ancestry has been taken that the remainder closes within the bound. Eligibility is not a refusal; it is deferral to a later round, by which time the greedy has taken the deeper ancestors on their own merits if they have any.

The depth bound serves the computation, not the economics. It limits the work of forming any one chunk and the imbalance between chunks on parallel lanes. A chain deeper than D that pays well at every level is taken level by level; a chain deeper than D whose fee is all at the leaf is never taken, which is the correct result, since the leaf cannot reach a block without ancestors no miner will take for free, and the relay rule's block scope [1] means no peer following it would have delivered such a leaf in the first place.

5.3 Prefix split at the block limit

A chunk larger than the remaining budget cannot be taken whole. It is split by prefix: its rows in ascending row order, which is topological, are taken up to the last that fits, and the prefix is a closed set because every row's parents precede it. The prefix is scored by its own fee and size, and competes on that basis.

Theorem 5.1 (Fee credit stays within a block)**.** *In a template built by prefix split, no transaction's fee is counted toward the inclusion of any row that is not in the same template.*

A chunk is taken whole or as a prefix, and in either case the rows taken and the fee counted are the same set. The suffix of a split chunk is not in the template and contributes no fee to it; in a later template, after the prefix confirms, the suffix's rows are a chunk of their own, scored by their own fee. The theorem is the mining counterpart of the pool's component cap, which refuses to credit a component with fee from an outcome that cannot be mined in one block. There, the point was that a fee the next block cannot collect is not one miner's prospect; here, the point is that the miner's packing never relies on such a fee either. A child's fee bumps its parents into *this* block or it does not bump them at all.

6 Packing

6.1 The greedy

The template is built by taking chunks in descending order of rate until nothing eligible fits. Taking a chunk marks its rows, and marked rows leave the ancestry of every descendant, so the descendants' chunks shrink and their rates change. The greedy proceeds in rounds:

1. *Score.* For every included, unmarked, eligible row, compute the fee and chargeable size of its chunk, and its rate.
2. *Select.* Visit roots in descending rate. For each, form the chunk exactly by walking untaken ancestry. If the chunk fits the remaining budget, take it and mark it; if it exceeds the budget, take the fitting prefix instead. If the walk meets a row marked earlier in this round, the root's score is stale: defer it.
3. *Repeat* until a round takes nothing, or the budget is exhausted.

The exact walk in the select step is what makes the template correct; the score is what makes it good. A score is computed once per round for every candidate root, and it may be stale by the time its root is visited, because an earlier take in the same round changed its ancestry. A stale root is deferred, not taken on a wrong score, and the next round rescores it. Taking a chunk only ever removes rows from a descendant's ancestry, so the number of rounds is bounded by the depth bound D, and in practice by the point at which the budget is spent, which is a few rounds for any pool with a block's worth of paying transactions.

6.2 What fits

A chunk fits when the template's weight and sigops, plus the chunk's, remain within the budget. Both are summed from the totals column, which holds virtual size and sigops per row. The budget is weight and sigops less the coinbase reserve. Nothing else is checked at packing: the included set is conflict-free and closed by resolution, every row is a validated transaction by admission, and the relative locks and maturity that depend on the next block's height and time were checked at admission against the pool context, which is the context of the block being built.

6.3 Order

The template's transaction order is the rows taken, in ascending row order. By the row-order lemma this places every parent before its children. It is not the order in which chunks were

taken; a chunk taken late may contain rows older than a chunk taken early, and interleaving them by row is still topological. No sort, no linearization and no dependency bookkeeping is performed. The template's `depends` list, for interfaces that report it [6], is read from the edges column.

6.4 Quality

The greedy over chunk rates is the standard approximation to the knapsack it replaces, and the approximation argument is the standard one: a greedy by density is optimal when the last item splits, and the prefix split makes the last chunk split at row granularity. The gap to optimal is at most the value of one chunk's worth of unsplit remainder, which is a few transactions against several thousand. The rescoring rounds recover the effect that bitcoind's ancestor-set selection recovers by updating descendants after each inclusion, and the cluster mempool recovers by maintaining linearizations: a parent taken on its own merit raises the rate of the children left behind. We do not claim the template is optimal; neither design does.

7 Algorithm

Every step above is a pass over the index's columns or a walk along its edges. The passes are the companion paper's passes with different parameters, and run in the same two forms: a *frontier* form that touches only the rows involved and is independent of pool size, and a *whole-column* form that touches every row and is a kernel over lanes or a grid.

7.1 Passes

Resolve.
One task per component. Enumerate outcomes by mixed radix over the cliques, score each by the criterion, and record the chosen outcome's mask. This is the admission evaluation with a different objective and no candidate, and it is bounded by the same member and outcome caps.

Exclude.
One pass over the cells. A member not in its component's chosen mask is excluded, and exclusion propagates to descendants along the children column to a fixed point. On termination the included set is marked.

Score.
One pass over the included rows. For each unmarked row, accumulate the fee and chargeable size of its untaken ancestry within the depth bound. In the frontier form this is a marking walk per row, exact. In the whole-column form it is a propagation along parent edges in row order, which is exact over trees and overcounts an ancestor shared by two parents, so it is a score, not a measurement; the exact figure is taken at select.

Select.
A threshold pass over rates fills a histogram and finds the rate above which the eligible chunks exceed the remaining budget; roots above threshold are gathered and visited in descending order. No full sort is needed; the template needs only the top of the distribution, and the next round moves the threshold down. The frontier form simply sorts the eligible roots by rate.

Take.
A marking walk per selected root over untaken ancestry, exact, with the fit test and prefix split. The walk is the relay rule's ancestry walk with the miner's budget in place of the peer's rate.

Resolve and exclude run once per template. Score, select and take run once per round. The marks are a scratch column of one byte per pool row, made for the template and dropped after it, and nothing persists between templates but the pool itself.

7.2 Two forms

The frontier form is what a CPU runs for one template. Resolve is a task per component over its members, exclude is a descent from the losers, score is a walk per eligible row and take is a walk per selected root. Its cost is proportional to the included rows and their ancestry, not to the pool. The whole-column form is what SIMD lanes or a GPU run. Each pass is a kernel over a column: exclusion is the pool's propagation kernel with a different seed, scoring is a gather along edges, and selection is a histogram and compaction. The take step is a walk in both forms, since it must be exact, but it runs only over the rows that end up in the block, a few thousand, and its cost is the same on any path.

The template function is pure. It reads the pool's columns and the criterion, and it writes a list of rows. It holds no state between calls, so there is nothing to invalidate at a new chain top, nothing to dump at shutdown and nothing to rebuild at start: the pool's own persistence [1] is the template's.

8 Templates in operation

8.1 When a template is built

A template is built on demand and against a chain top. The events that change the correct template are a new confirmed block, which removes the confirmed rows and their conflicts from the pool and may unconfirm others; a reorganization, which does both in the other direction; and admission, which adds rows. The pool's index is updated by each of these through the events the archive already sees, and the template is derived from the index, so no template-specific maintenance accompanies them. A miner's template at a given top is recomputed, not patched, and it is recomputed as often as the miner asks: once per block, on an interval to pick up new admissions, or whenever a high-fee admission is seen, which the transaction chaser can signal since it holds the fee at admission.

The cost figures of Section 10 are what make this policy reasonable. A template that costs milliseconds is cheaper to rebuild than to maintain, and a template that is a pure function of the index cannot be wrong about what the index holds.

8.2 Output

The template is delivered as the ordered list of transactions with, per transaction, its fee, weight and sigops from the totals column and its pool dependencies from the edges column, and as a whole its total fee, weight and sigops, the coinbase reserve it left, and the top against which it was built. These are the fields the `getblocktemplate` interface reports [6, 7], and the node's bitcoind interface serves them from the template object. The coinbase, its witness commitment and the block header are the miner's, as they are everywhere.

8.3 Multiple criteria

Because the criterion is a parameter and the template is a pure function, a node can build several templates against one top under several criteria at the cost of several runs, or expose the criterion to the caller. A pool operator who wants the fee-maximal template and a rate-maximal alternative gets both; a miner who wants to see what its revenue would be under first-seen resolution, for comparison with bitcoind's behavior, can ask for that too. None of this touches the pool.

9 Comparison with bitcoind

9.1 Ancestor-set selection

bitcoind's block assembler [4] iterates over mempool entries by ancestor fee rate, the fee of an entry plus its unconfirmed ancestors over their combined size, includes the entry with its ancestors when they fit, and updates the ancestor state of the entry's descendants before continuing. The ancestor and descendant sets are maintained per entry at admission, which is also what bounds them: the ancestor and descendant limits of 25 transactions and 101 kvB exist so that the maintenance stays cheap. The mempool is conflict-free, so the assembler never resolves anything.

9.2 The cluster mempool

The cluster mempool [5] partitions the mempool into clusters connected by dependency, maintains for each a linearization chunked so that chunk rates are non-increasing, and builds a template by merging chunks across clusters in rate order until the block fills. Template construction is fast because the linearizations were paid for at admission. The mempool is still conflict-free; replacement is the resolution, decided at admission by comparing fee-rate diagrams.

9.3 Three differences

Where conflicts are resolved.
bitcoind resolves at admission, once, for every miner that will ever draw on that node's mempool, by a policy that is not the miner's. We resolve at template time, per template, by the miner's criterion. The pool holds what bitcoind threw away, and the miner sees it.

Where order is produced.
bitcoind computes an ordering, by ancestor score or by linearization, and maintains it. We read an ordering from row order, which is topological by construction, and produce nothing but the choice of rows. The only ordering work in the design is the descending visit of chunk rates within a round.

What persists between templates.
bitcoind keeps per-entry ancestor and descendant state, or per-cluster linearizations, updated at every admission so that the template is cheap. We keep the pool's index, which admission maintains for its own reasons, and derive the template from it each time. The template has no state, and the admission path carries no cost on the template's behalf.

9.4 Consequences

The admission limits that bitcoind imposes to keep template state maintainable, ancestor and descendant counts and sizes, have no counterpart here. The pool caps contested components for the fee-credit reason and bounds nothing else; a chain of a thousand paying transactions is admitted and mined level by level. The cluster mempool's cluster size limit, likewise, exists so that linearization is tractable; our resolution enumerates a bounded component and our packing walks only what is taken, so the pool size that matters to the template is the included set's, not the cluster's.

The template's cost is paid when a template is wanted, by the party that wants it, rather than spread over every admission by every node. A relay node that never mines pays nothing for templates. A miner pays milliseconds per template, on hardware the miner chooses.

10 Cost estimates

All figures here are estimates from bandwidth, instruction throughput and algorithm structure, on the companion paper's reference machine: a 64-core Zen 5 workstation with full-width

AVX-512 and 512 GB of DDR5-6400, and a Blackwell workstation GPU with 96 GB, with a pool of one million rows resident in memory. None is a measurement. One whole-column gather pass is about 10 ms on one scalar core, 5 ms on one AVX-512 core, 0.2 ms on 64 cores and 20 μs on the GPU; a streaming pass is roughly a thirtieth of that [1].

10.1 Three pool shapes

Quiet.
Few contests; components small; most rows uncontested in chains of depth two or three; a block's worth of paying rows near the top of the rate distribution. Two rounds fill the budget.

Typical.
Several thousand components of medium complexity, about ten members and thirty outcomes each; chains to depth ten; four rounds.

Contested.
Components at the member bound with the outcome cap, numbering in the thousands; chains to the depth bound; eight rounds.

10.2 Per template

Resolution is one enumeration per component, from 1 μs for a trivial component to about 0.3 ms for one at both caps on one core, and all components run at once. Exclusion is one propagation to a fixed point, a few passes. A round is one score pass, one streaming threshold pass and the takes, which total a few thousand walks of depth bounded by D and cost about the same on every path, on the order of a millisecond.

shape	rounds	frontier, 1 core	AVX-512, 1 core	64 cores	GPU
quiet	2	15–30 ms	15–25 ms	2–3 ms	1–1.5 ms
typical	4	40–80 ms	35–60 ms	3–5 ms	1.5–2.5 ms
contested	8	0.3–0.8 s	0.15–0.3 s	8–15 ms	3–5 ms

Table 1: Estimated cost of one template over a one-million row pool. The frontier column walks only the included rows and their ancestry; the others run whole-column passes per round. The GPU figure includes the take walks on the host, which dominate it.

The frontier figure for the contested shape is dominated by resolution of thousands of components at the outcome cap on one core; the same work on 64 cores is a task per component and is why that column barely moves between shapes. The GPU figure is dominated by the take walks, which are host work regardless, so the device buys about an order of magnitude over 64 cores on the passes and nothing on the takes; a device-side take over the compacted roots would remove that floor.

10.3 bitcoind at the same shapes

The honest comparison is on one core, frontier against bitcoind: the ancestor-set assembler scales with entries times inclusions and is slow at a million entries, which is why bitcoind's default mempool is a fraction of that size; the cluster mempool's merge is fast because its cost was moved to admission, and is comparable to our frontier form on the quiet and typical shapes while never having to resolve anything, because it never holds anything to resolve. On 64 cores or the GPU we are an order of magnitude or more under either, for the reason the companion paper gives for admission: the work is column passes, and column passes scale with lanes.

design	work at one million entries	estimate
ancestor-set assembler	iterate by ancestor score, include with ancestors, update descendants; indexed set operations per inclusion	0.5–2 s
cluster mempool template	merge maintained chunks across clusters by rate until full; heap over clusters	10–50 ms

Table 2: Estimated template cost in bitcoind at one million mempool entries, one core, scalar. The cluster mempool's figure excludes the linearization cost it paid at admission, which the companion paper counts there.

11 Open items

The version 5 metadata graph supplies the object model: a template object holding its rows, totals and top; a criterion object the miner supplies; and the interface through which both are exposed. Nothing in this paper's computation waits on it.

The score pass in whole-column form overcounts ancestors shared by two parents of one row. The take is exact, so templates are correct; the cost is a mis-ranked chunk where a diamond is present. An exact whole-column score is a set union per row and does not vectorize; the frontier form's per-row walk is exact at a cost proportional to ancestry, which the depth bound keeps small. Which form a node runs for scoring is a measurement to make, not a decision to take now.

A device-side take, walking compacted roots on the GPU with an atomic budget, would remove the host floor from the GPU figures. It is an optimization of a step that already costs a millisecond.

The template is a pure function of the pool and the top. A miner that wants a template immediately at a new top, before the pool's confirm pass has settled, can be given the previous template less the confirmed rows, which is correct but not best, and then the rebuilt one; the pool paper's confirmation events make the first available at once.

References

[1] E. Voskuil, *Tidal Pool: An Objective, Scalable, Semi-Persistent Bitcoin Transaction Pool*, 12 November 2026. Companion to this paper.

[2] E. Voskuil. *Libbitcoin Version 4 Node*, version 8, 6 November 2021. Published on the project site: `https://libbitcoin.org/architecture/node/libbitcoin-v4-node-design.pdf`. Cited passages: optimization goals (p. 16), transaction pool and double spending (pp. 13–14).

[3] E. Voskuil and P. Mienk, *Narrowcast Transaction Relay: Adaptive Packaging for Bitcoin Transaction Propagation*, August 2022, with a note added in proof, August 2026.

[4] S. Daftuar, *Select transactions using feerate with ancestors*, bitcoind block assembler, 2016; the ancestor-set selection used by `CreateNewBlock` since version 0.13.

[5] S. Daftuar and P. Wuille, *Cluster mempool*, design proposal and bitcoind implementation, 2023–2025; P. Wuille, *Spanning-forest cluster linearization.*

[6] L. Dashjr, *BIP 22: getblocktemplate — Fundamentals*, 2012.

[7] L. Dashjr, *BIP 23: getblocktemplate — Pooled Mining*, 2012.

[8] E. Lombrozo, J. Lau and P. Wuille, *BIP 141: Segregated Witness (Consensus layer)*, 2015: block weight and sigop cost.